

TaskFlow

Séance 4 : UI Libraries & Architecture

Réponses aux questions

Table des matières

1	Partie 1 : Header avec Material UI	2
2	Partie 2 : Header MUI vs Header Bootstrap	3
3	Partie 3 : Système de style	4
4	Partie 4 : Tableau comparatif	5
5	Partie 5 : Architecture Base de Données	6
5.1	Architecture actuelle de TaskFlow	6
5.2	Architecture avec Firebase	6
5.3	Architecture avec Express + MongoDB	7
6	Partie 6 : Questions de réflexion	8

1 Partie 1 : Header avec Material UI

Q1 : Combien de lignes de CSS avez-vous écrit pour le Header MUI? Comparez avec votre Header.module.css.

Réponse :

Pour le Header MUI, **0 ligne de CSS** a été écrite. Tout le style est géré via la prop **sx** directement dans le composant React.

TABLE 1 – Comparaison des headers

Header	Lignes de CSS	Méthode
Original (Header.module.css)	Environ 50	Fichier CSS séparé
MUI (HeaderMUI.tsx)	0	Style inline via sx

Avantages de MUI :

- Pas de fichier CSS séparé
- Styles co-localisés avec le composant
- Thème unifié et cohérent
- Moins de code à maintenir

Inconvénients :

- Le code JSX devient plus verbeux
- Moins de séparation des préoccupations

2 Partie 2 : Header MUI vs Header Bootstrap

Q2 : Comparez le code du Header MUI vs Bootstrap. Lequel est plus lisible ? Plus court ?

Réponse :

TABLE 2 – Comparaison de longueur de code

Composant	Lignes	Observations
Header MUI	~25	Code plus verbeux à cause de <code>sx</code>
Header Bootstrap	~20	Code plus concis avec classes CSS

TABLE 3 – Comparaison de lisibilité

Critère	Material UI	React-Bootstrap
Syntaxe	JSX avec <code>sx</code>	JSX avec classes Bootstrap
Courbe d'apprentissage	Élevée	Faible
Débogage	Complexe	Simple
Maintenabilité	Bonne	Bonne

Conclusion :

- **Bootstrap** est plus **court et lisible** car il utilise des classes CSS standard comme `bg="success", variant="outline-light"`.
- **MUI** offre plus de **puissance et flexibilité** mais au prix d'un code plus verbeux.

3 Partie 3 : Système de style

Q3 : Le Login MUI utilise `sx{}` pour le style. Le Login Bootstrap utilise des classes CSS (`className`). Quel système préférez-vous ? Pourquoi ?

Réponse :

Je préfère le système de classes CSS (Bootstrap) pour les raisons suivantes :

1. **Séparation des préoccupations** : Le style est séparé du code JavaScript.
2. **Lisibilité immédiate** : Les classes comme `d-flex`, `justify-content-center` sont immédiatement compréhensibles.
3. **Réutilisabilité** : Les classes CSS peuvent être réutilisées dans plusieurs composants.
4. **Performance** : Les classes CSS sont optimisées et mises en cache.
5. **Débogage** : Plus facile dans les DevTools.

TABLE 4 – Comparaison des systèmes de style

Critère	sx (MUI)	Classes CSS (Bootstrap)
Séparation des préoccupations		
Lisibilité	Objet JS long	Classes explicites
Réutilisabilité	Duplication	Classes réutilisables
Performance	Style inline	CSS optimisé
Débogage	Moins facile	Très facile

4 Partie 4 : Tableau comparatif

TABLE 5 – Material UI vs React-Bootstrap

Critère	Material UI	React-Bootstrap
Installation	<code>npm install @mui/material @emotion/react @emotion/styled @mui/icons-material</code>	<code>npm install react-bootstrap bootstrap</code>
Composants utilisés	6 (AppBar, Toolbar, Typography, IconButton, Button, Box)	5 (Navbar, Container, Button, Nav, Navbar.Brand)
Lignes CSS écrites	0	0
Système de style	CSS-in-JS (prop <code>sx</code>)	Classes CSS (Bootstrap)
Personnalisation	Très facile	Modérée
Responsive	Oui	Oui
Lisibilité	Bonne	Très bonne
Documentation	Excellente	Excellente
Préférence	Projets complexes	Prototypes rapides

Q4 : Si vous deviez choisir UNE seule library pour TaskFlow en production, laquelle et pourquoi ?

Réponse :

Je choisirais **Material UI** pour les raisons suivantes :

1. **Personnalisation plus poussée** : Système de thème complet
2. **Composants plus riches** : DataGrid, DatePicker, etc.
3. **Meilleure intégration React** : CSS-in-JS co-localisé
4. **Accessibilité** : ARIA, support clavier

5 Partie 5 : Architecture Base de Données

5.1 Architecture actuelle de TaskFlow

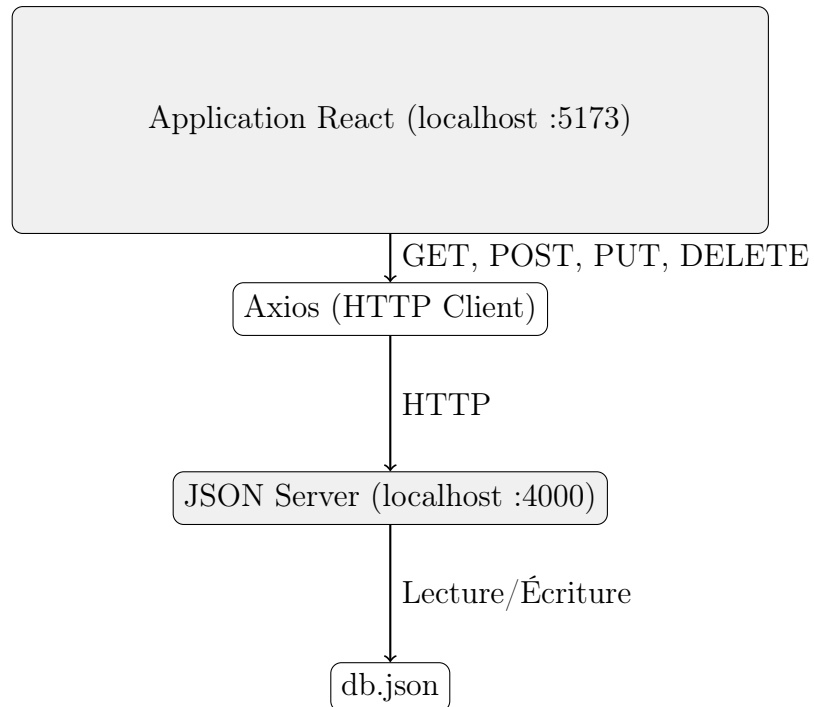


FIGURE 1 – Architecture actuelle de TaskFlow

5.2 Architecture avec Firebase

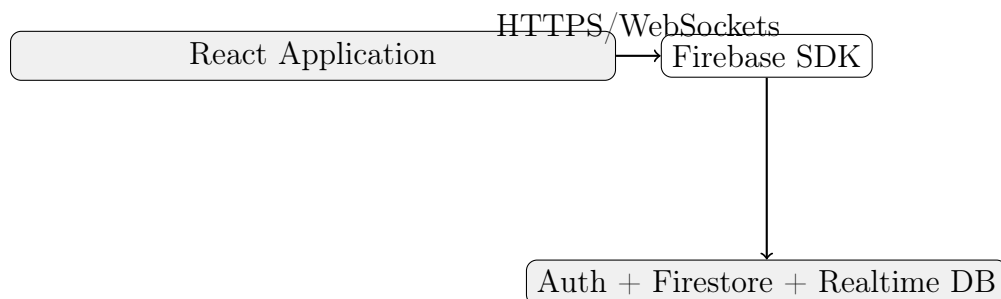


FIGURE 2 – Architecture avec Firebase

5.3 Architecture avec Express + MongoDB

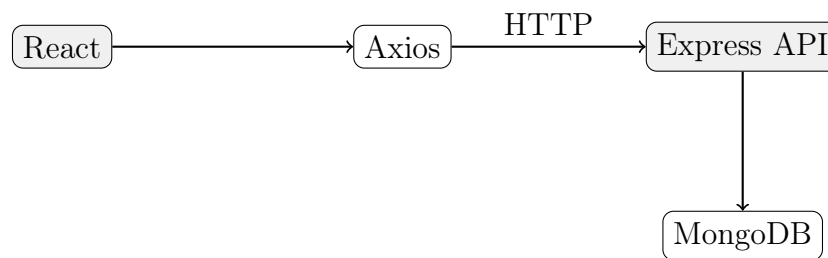


FIGURE 3 – Architecture avec Express + MongoDB

Q5 : Pourquoi React ne peut-il PAS se connecter directement à MySQL ?

Réponse :

React s'exécute dans le navigateur client. Une connexion directe à MySQL nécessiterait d'exposer les identifiants de la base de données dans le code source côté client, ce qui représente un risque de sécurité majeur (injection SQL, accès non autorisé).

Q6 : json-server est parfait pour notre TP. Donnez 3 raisons pour lesquelles on ne l'utiliserait PAS en production.

Réponse :

1. **Aucune sécurité** : Pas d'authentification, pas de validation des données
2. **Persistance limitée** : Fichier JSON non adapté aux écritures concurrentes
3. **Performances** : Ne supporte pas un grand nombre d'utilisateurs simultanés

Q7 : Firebase permet à React de se connecter directement (pas de backend Express). Comment est-ce possible alors que MySQL ne le permet pas ?

Réponse :

Firebase est un **BaaS (Backend as a Service)** qui fournit :

- Des SDK sécurisés avec règles de sécurité côté serveur
- Une authentification intégrée avec tokens JWT
- Des règles Firestore pour contrôler les accès
- Une API REST ou WebSockets qui encapsulent l'accès

Les identifiants sont gérés par Firebase, non exposés dans le code client.

6 Partie 6 : Questions de réflexion

Q8 : Votre TaskFlow utilise json-server. Un client vous demande de passer en production avec de vrais utilisateurs. Quelles étapes sont nécessaires ?

Réponse :

1. Créer un backend avec Node.js/Express
2. Choisir une base de données (PostgreSQL, MongoDB, MySQL)
3. Migrer les données de `db.json` vers la vraie BDD
4. Implémenter l'authentification avec JWT et bcrypt
5. Sécuriser les routes avec des middlewares d'authentification
6. Déployer le backend sur un serveur (Heroku, Railway, AWS)
7. Modifier l'URL de l'API dans `axios.ts`
8. Configurer CORS
9. Déployer le frontend sur Netlify/Vercel

Q9 : MUI et Bootstrap sont des libraries externes. Quel est le risque d'en dépendre ?

Réponse :

1. **Taille du bundle** : Augmentation significative (MUI ~120KB, Bootstrap ~50KB + CSS)
2. **Breaking changes** : Les mises à jour majeures peuvent casser l'application
3. **Dépendance** : La maintenance dépend de la pérennité de la librairie
4. **Personnalisation limitée** : Difficile à surcharger parfois

Q10 : Vous devez créer une app de chat en temps réel. json-server, Firebase ou Backend custom ? Justifiez.

Réponse :

Je choisirais **Firebase** pour les raisons suivantes :

1. **Temps réel natif** : Support des WebSockets et synchronisation en temps réel
2. **Authentification intégrée** : Gère facilement les utilisateurs
3. **Simplicité** : Pas besoin de développer un backend complexe
4. **Scalabilité** : Gère automatiquement un grand nombre de connexions
5. **Coût** : Offre un généreux quota gratuit

Pourquoi pas json-server ? Ne supporte pas le temps réel et ne scale pas.

Pourquoi pas backend custom ? Plus complexe à développer et maintenir.

Conclusion

TP Séance 4 complété

Le projet TaskFlow intègre désormais deux librairies UI (Material UI et React-Bootstrap) avec une architecture bien documentée.